Department of Information Technology

Software Engineering and Project Management

Smart Student Task & Assignment Tracking System

Project Proposal and Software Engineering Documentation

Student Name:	Md. Mashrafi Rahman
Student ID:	IT-24034
Course:	SEPM-ICT-3107
Project Type:	Individual Software Engineering Project
Development Model:	Agile Scrum
Programming Language:	Java 26
GUI Technology:	Java Swing + FlatLaf
Database:	SQLite
IDE:	IntelliJ IDEA

October 27, 2026

Contents

Abstract **3**

1 Project Idea and Requirements **3**
1.1 Project Title . . . 3
1.2 Project Idea . . . 3
1.3 Problem Statement . . . 3
1.4 Proposed Solution . . . 4

2 Project Proposal **4**
2.1 Background . . . 4
2.2 Project Goal . . . 4
2.3 Project Objectives . . . 4
2.4 Target Users . . . 5

3 Project Scope **5**
3.1 In Scope . . . 5
3.2 Out of Scope . . . 6

4 Functional Requirements **6**
4.1 Non-Functional Requirements . . . 6

5 System Features **7**
5.1 Dashboard . . . 7
5.2 Course Management . . . 7
5.3 Task Management . . . 7
5.4 Assignment Management . . . 7

6 Software Development Process Model **8**
6.1 Agile Scrum . . . 8
6.2 Why Agile Scrum? . . . 8

7 Project Schedule **8**

8 Project Milestones **9**

9 Project Deliverables **9**

10 Gantt Chart **10**

11 Data Flow Diagram **10**

12 Use Case Diagram **10**

13 Activity Diagram **11**

14 Entity Relationship Diagram **12**

15 Class Diagram **13**
15.1 Class Responsibilities 14

16 Object Diagram **14**

17 Graphical User Interface Design **15**
17.1 Main Dashboard Prototype 15

18 Java Implementation Design **15**
18.1 Technology Stack 16
18.2 Proposed Java Project Structure 16

19 Database Design **17**
19.1 Proposed Tables 17
19.2 Database Flow 17

20 Testing Plan **17**

21 Risk Analysis **18**

22 Quality Assurance **18**

23 Future Improvements **18**

24 Expected Outcome **19**

25 GitHub Repository Structure **19**

26 Final Submission Checklist **20**

27 Conclusion **21**

Student Information **22**

Abstract

The Smart Student Task & Assignment Tracking System is a Java-based desktop application designed to help students organize and monitor their academic activities from a centralized platform. The system allows students to manage courses, tasks, assignments, deadlines, priorities, completion status, and academic progress.

The project follows the Agile Scrum software development process. The system is designed using standard software engineering practices including requirements analysis, UML modelling, database design, interface design, implementation, testing, and documentation.

The application is implemented using Java 26, Java Swing with FlatLaf for the graphical user interface, SQLite for persistent data storage, Maven for dependency management, and IntelliJ IDEA as the development environment.

1 Project Idea and Requirements

1.1 Project Title

Smart Student Task & Assignment Tracking System

1.2 Project Idea

The proposed system is a student-oriented academic task management application. Students often manage academic activities using notebooks, messaging applications, calendars, or different productivity applications. This can make it difficult to maintain a clear overview of assignments, tasks, deadlines, and academic progress.

The proposed system provides a centralized desktop application where students can record courses, create academic tasks and assignments, specify deadlines and priorities, update completion status, and monitor their progress.

1.3 Problem Statement

Students may face the following problems:

- Difficulty remembering multiple assignment deadlines.
- Lack of centralized task management.
- Difficulty identifying high-priority academic work.
- Poor visibility of completed and pending tasks.
- Academic information being distributed across different tools.
- Difficulty monitoring overall academic progress.

1.4 Proposed Solution

The proposed system provides one centralized platform for academic task and assignment management.

The system will allow a student to:

1. Manage courses.
2. Create and manage academic tasks.
3. Create and manage assignments.
4. Set priorities.
5. Set deadlines.
6. Mark tasks as completed.
7. View pending and completed work.
8. Monitor academic progress.
9. Search and filter academic activities.

2 Project Proposal

2.1 Background

Modern students are required to handle multiple academic activities simultaneously. Courses may contain assignments, projects, presentations, laboratory tasks, quizzes, examinations, and other academic requirements. Without an organized tracking mechanism, students may miss deadlines or lose track of their progress.

The Smart Student Task & Assignment Tracking System aims to provide a simple and centralized solution specifically focused on academic activities.

2.2 Project Goal

The main goal is to develop a user-friendly Java desktop application that allows students to efficiently organize academic tasks and assignments and monitor their progress.

2.3 Project Objectives

The main objectives are:

- To develop a centralized academic task management system.
- To provide a simple graphical user interface.
- To manage courses and academic activities.
- To track assignment deadlines.

- To provide task priority and completion status.
- To maintain academic records using SQLite.
- To demonstrate object-oriented programming concepts.
- To apply software engineering principles throughout development.
- To design the system using UML and database modelling.
- To develop and test a functional prototype.

2.4 Target Users

The primary target user is:

- University students.

Future versions may support:

- Teachers.
- Academic advisors.
- Administrators.

3 Project Scope

3.1 In Scope

The first version will include:

- Student dashboard.
- Course management.
- Task management.
- Assignment management.
- Deadline management.
- Priority management.
- Completion status.
- Progress monitoring.
- Search and filtering.
- SQLite database.
- Desktop GUI.

3.2 Out of Scope

The initial version will not focus on:

- Online examination.
- Video conferencing.
- University-wide student administration.
- Online payment.
- Cloud synchronization.

These features may be considered in future versions.

4 Functional Requirements

ID	Requirement	Description
FR-01	Login	The student can access the system.
FR-02	Dashboard	The system displays an overview of academic activities.
FR-03	Course Management	The student can add, edit, delete and view courses.
FR-04	Task Management	The student can create, edit, delete and complete tasks.
FR-05	Assignment Management	The student can manage academic assignments.
FR-06	Deadline Tracking	The system stores and displays deadlines.
FR-07	Priority Management	Tasks can have different priority levels.
FR-08	Progress Tracking	The system displays completion progress.
FR-09	Search	Users can search academic activities.
FR-10	Filtering	Users can filter tasks according to status or priority.

4.1 Non-Functional Requirements

- **Usability:** The interface should be simple and understandable.
- **Performance:** Common operations should respond quickly.
- **Reliability:** Data should be stored persistently.
- **Maintainability:** The system should follow modular OOP design.
- **Portability:** The project should run on Java-compatible systems.
- **Security:** User data should not be exposed unnecessarily.
- **Scalability:** The architecture should allow future features.

5 System Features

5.1 Dashboard

The dashboard provides a summary of:

- Total courses.
- Pending tasks.
- Completed tasks.
- Pending assignments.
- Upcoming deadlines.
- Overall progress.

5.2 Course Management

Students can create and manage course information such as:

- Course code.
- Course title.
- Instructor.
- Semester.

5.3 Task Management

A task may contain:

- Task title.
- Description.
- Course.
- Priority.
- Deadline.
- Status.

5.4 Assignment Management

Assignments can contain:

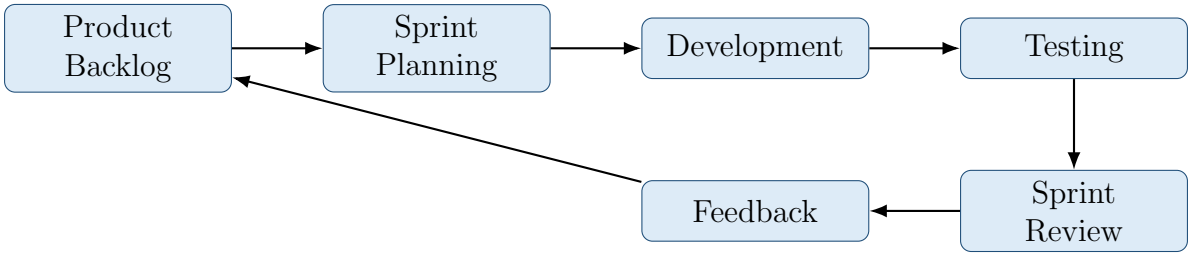
- Assignment title.
- Course.
- Description.
- Submission deadline.
- Status.

6 Software Development Process Model

6.1 Agile Scrum

The project will use the Agile Scrum methodology.

Agile allows the system to be developed incrementally through multiple short development cycles called sprints.



6.2 Why Agile Scrum?

Agile Scrum is suitable because:

- Requirements can be refined during development.
- Features can be implemented incrementally.
- Testing can occur throughout development.
- Feedback can be incorporated into future iterations.
- A working prototype can be produced early.

7 Project Schedule

The proposed project duration is ten weeks.

Week	Activity	Milestone	Deliverable
1	Requirements analysis	M1	Requirements document
2	Proposal development	M2	Initial proposal
3	UML design	M3	UML diagrams
4	Database design	M4	ERD and database schema
5	GUI design	M5	GUI prototype
6	Core Java development	M6	Working application
7	SQLite integration	M7	Database-enabled system
8	Feature completion	M8	Functional prototype
9	Testing	M9	Test report
10	Documentation and submission	M10	Final project package

8 Project Milestones

1. **M1:** Requirements completed.
2. **M2:** Project proposal completed.
3. **M3:** UML diagrams completed.
4. **M4:** Database design completed.
5. **M5:** GUI prototype completed.
6. **M6:** Core Java implementation completed.
7. **M7:** SQLite integration completed.
8. **M8:** Functional prototype completed.
9. **M9:** Testing completed.
10. **M10:** Final documentation and GitHub submission completed.

9 Project Deliverables

The project will produce the following deliverables:

- Project proposal.
- Requirements specification.
- DFD.
- Use Case Diagram.
- Activity Diagram.
- Entity Relationship Diagram.
- Class Diagram.
- Object Diagram.
- Gantt Chart.
- Java source code.
- SQLite database.
- Graphical user interface.
- Test cases and results.
- Final README.
- Final PDF documentation.
- GitHub repository.

10 Gantt Chart

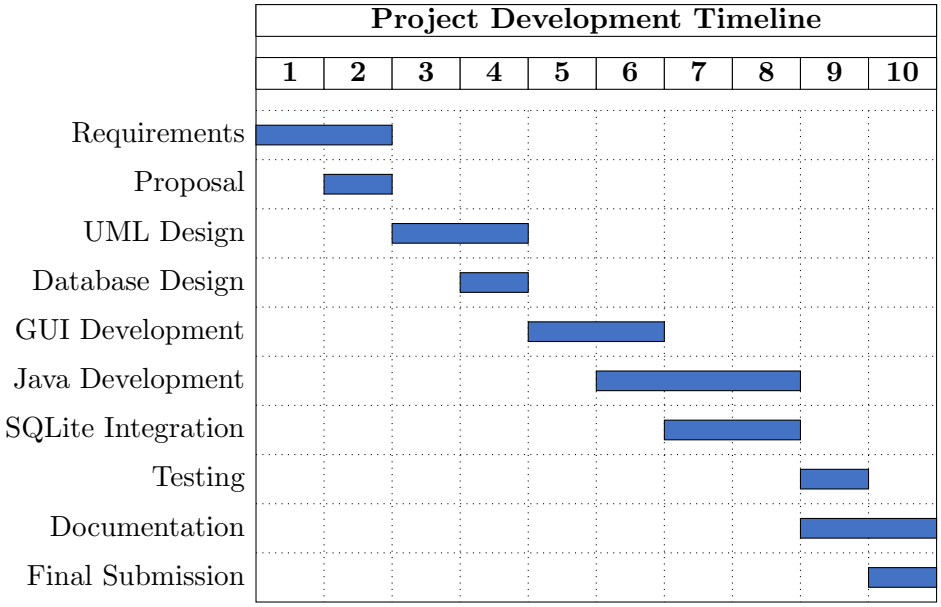


Figure 1: Project Gantt Chart

11 Data Flow Diagram

The Data Flow Diagram shows how information moves between the student, application processes, and database.

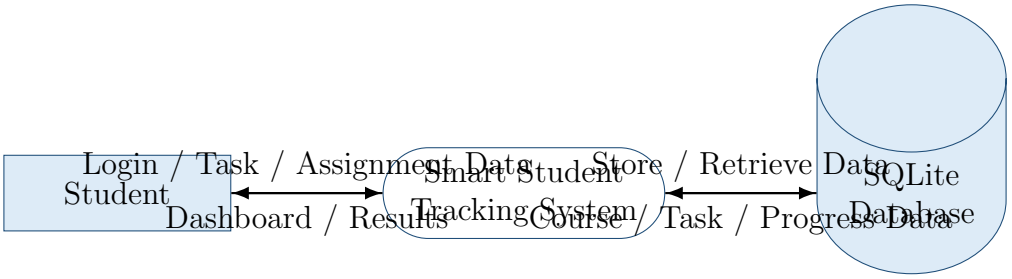


Figure 2: Level-0 Data Flow Diagram

12 Use Case Diagram

The main actor in the initial version is the Student.

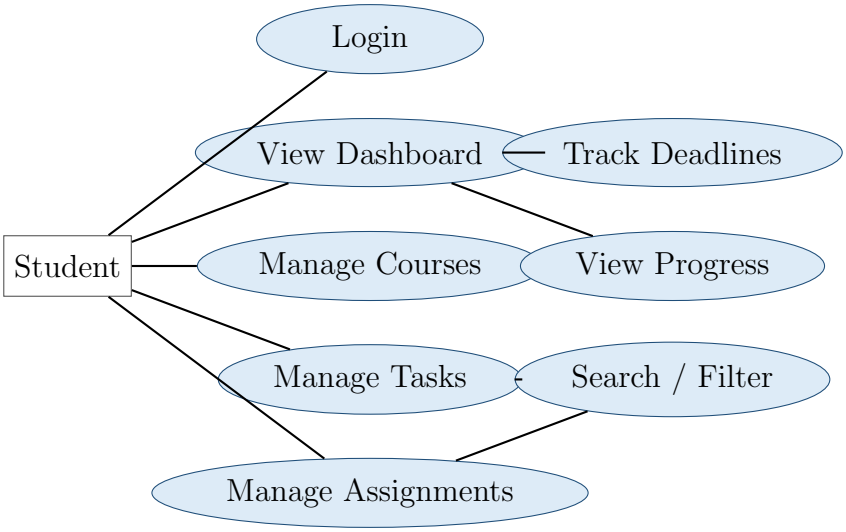


Figure 3: Use Case Diagram

13 Activity Diagram

The following activity diagram represents the process of creating a new academic task.

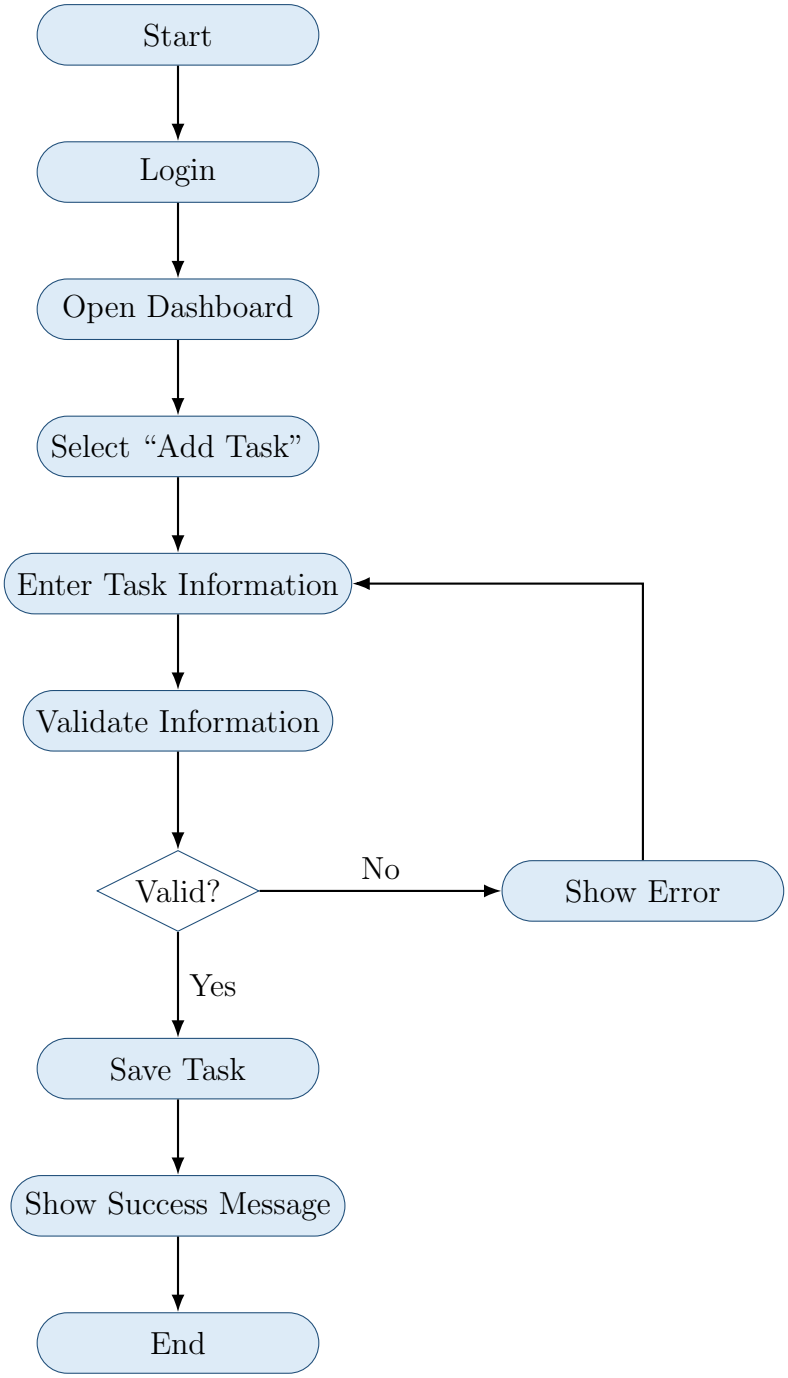


Figure 4: Activity Diagram for Creating a Task

14 Entity Relationship Diagram

The initial database design contains the following entities:

- Student
- Course
- Task
- Assignment

- Progress

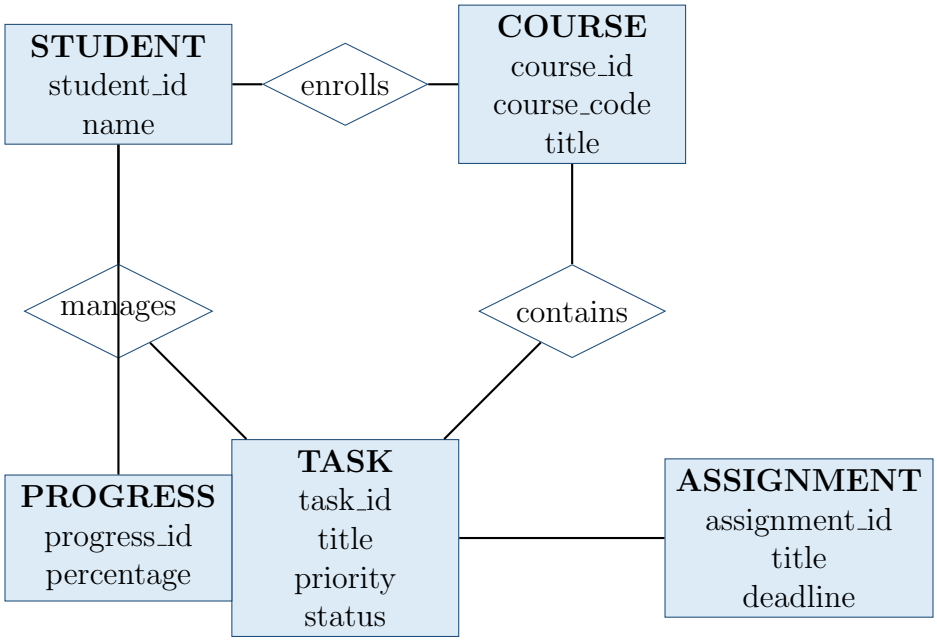


Figure 5: Entity Relationship Diagram

15 Class Diagram

The Java implementation will use object-oriented classes corresponding to the system design.

The main classes include:

- User
- Student
- Teacher
- Course
- Task
- Assignment
- Progress

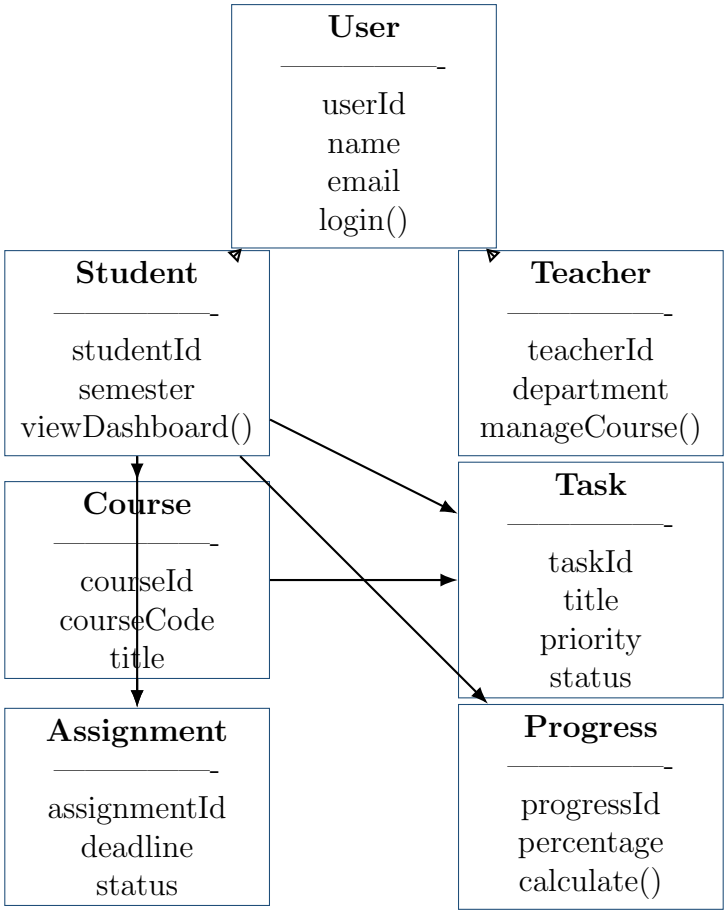


Figure 6: Class Diagram

15.1 Class Responsibilities

Class	Responsibility
User	Stores common user information.
Student	Represents a student using the system.
Teacher	Represents academic instructor information.
Course	Represents an academic course.
Task	Represents a general academic task.
Assignment	Represents a course assignment.
Progress	Stores and calculates academic progress.

16 Object Diagram

The Object Diagram represents example runtime objects created from the classes.

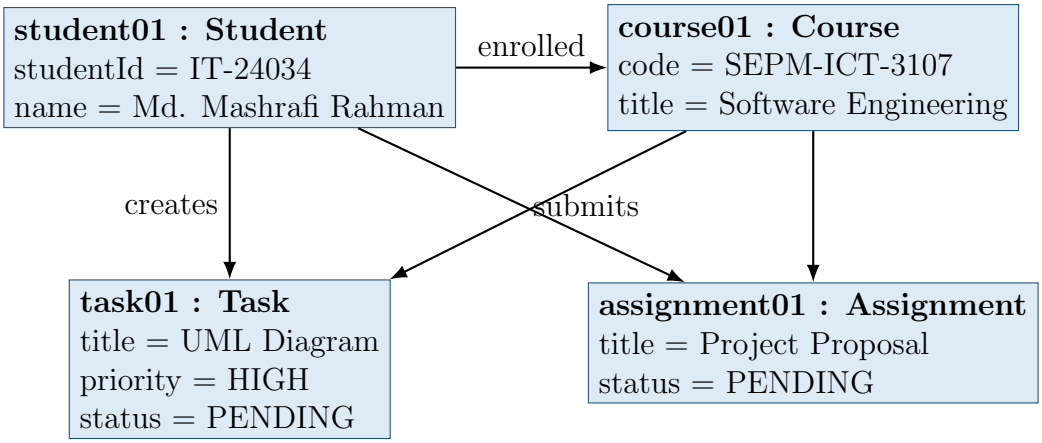


Figure 7: Example Object Diagram

17 Graphical User Interface Design

The application will use Java Swing with FlatLaf to provide a modern desktop interface.

17.1 Main Dashboard Prototype

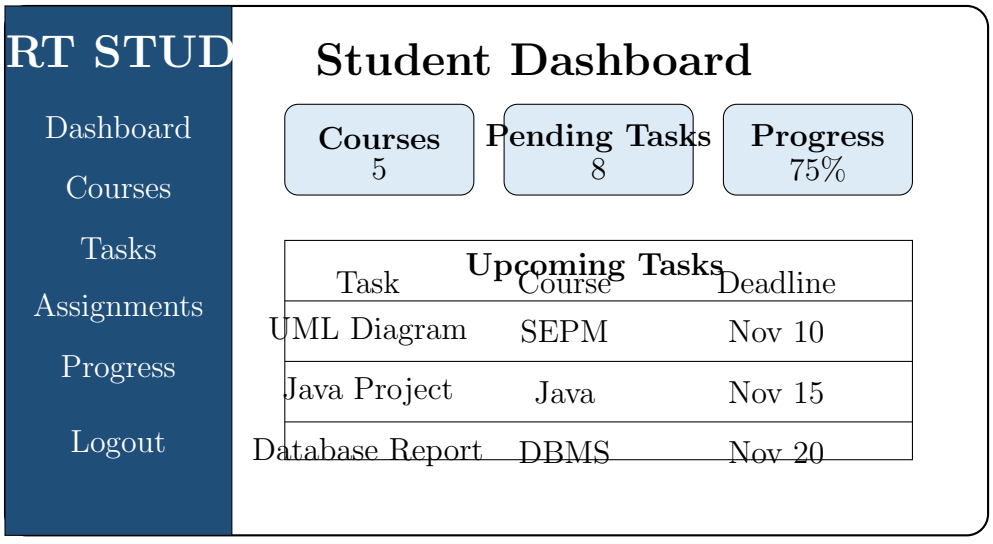


Figure 8: Conceptual Dashboard Interface

18 Java Implementation Design

The system will be implemented using Java 26.

18.1 Technology Stack

Technology	Purpose
Java 26	Application programming language
Java Swing	Desktop graphical interface
FlatLaf	Modern Swing user interface appearance
SQLite	Local persistent database
Maven	Dependency and project management
IntelliJ IDEA	Development environment
Git/GitHub	Version control and project submission

18.2 Proposed Java Project Structure

```
Smart-Student-Task-Tracking-System/
|
+-- src/
| +-- main/
| +-- java/
| +-- com/mashrafi/smartstudent/
| |
| +-- App.java
| |
| +-- model/
| | +-- User.java
| | +-- Student.java
| | +-- Teacher.java
| | +-- Course.java
| | +-- Task.java
| | +-- Assignment.java
| | +-- Progress.java
| |
| +-- data/
| | +-- Database.java
| |
| +-- ui/
| +-- MainFrame.java
| +-- DashboardPanel.java
| +-- CoursePanel.java
| +-- TaskPanel.java
| +-- AssignmentPanel.java
|
+-- pom.xml
+-- README.md
+-- smart_student.db
```

19 Database Design

SQLite will be used as the local database.

19.1 Proposed Tables

Table	Purpose
students	Stores student information.
teachers	Stores teacher information.
courses	Stores course information.
tasks	Stores academic tasks.
assignments	Stores assignments and deadlines.
progress	Stores progress information.

19.2 Database Flow



20 Testing Plan

Testing will be performed after implementation.

ID	Test Case	Expected Result	Status
TC-01	Application startup	Main window opens successfully	Pending
TC-02	Add course	Course is stored in database	Pending
TC-03	Edit course	Course information is updated	Pending
TC-04	Delete course	Course is removed	Pending
TC-05	Add task	Task appears in task list	Pending
TC-06	Complete task	Task status changes to completed	Pending
TC-07	Add assignment	Assignment is stored	Pending
TC-08	Deadline display	Upcoming deadlines are shown	Pending
TC-09	Search task	Matching tasks are displayed	Pending
TC-10	Database persistence	Data remains after restart	Pending

21 Risk Analysis

Risk	Description	Impact	Mitigation
R1	Database connection problems	Medium	Test database layer separately
R2	GUI complexity	Medium	Use modular Swing panels
R3	Development delay	High	Follow Agile milestones
R4	Data loss	High	Use SQLite transactions and backups
R5	Integration errors	Medium	Integrate features incrementally
R6	Requirement changes	Medium	Maintain flexible Agile backlog

22 Quality Assurance

The project will follow the following quality practices:

- Modular object-oriented design.
- Meaningful class and variable names.
- Separation of GUI and database logic.
- Validation of user input.
- Database persistence testing.
- Functional testing.
- Regression testing after major changes.
- Git version control.
- Documentation of important classes and features.

23 Future Improvements

Future versions may include:

- Cloud synchronization.
- Mobile application.
- Web application.
- Teacher dashboard.
- Administrator dashboard.
- Email notifications.
- Calendar integration.

- Automatic deadline reminders.
- AI-based task prioritization.
- Academic performance analytics.
- Multi-device synchronization.

24 Expected Outcome

At the end of the project, the expected outcome is a functional Java desktop application that allows students to manage academic tasks, assignments, courses, deadlines, and progress from one centralized interface.

The project will also demonstrate practical application of:

- Object-oriented programming.
- Software engineering principles.
- Agile development.
- UML modelling.
- Database design.
- GUI development.
- Software testing.
- Project management.

25 GitHub Repository Structure

The final GitHub repository will be organized as follows:

```
Smart-Student-Task-Tracking-System/
|
+-- README.md
|
+-- Proposal/
| +-- Project-Proposal.tex
| +-- Project-Proposal.pdf
|
+-- UML/
| +-- DFD.png
| +-- Use-Case.png
| +-- Activity-Diagram.png
| +-- ERD.png
| +-- Class-Diagram.png
| +-- Object-Diagram.png
|
+-- Gantt/
```

```
| +-- Gantt-Chart.png
|
+-- src/
| +-- main/
| +-- java/
|
+-- database/
| +-- smart_student.db
|
+-- screenshots/
|
+-- test/
|
+-- pom.xml
```

26 Final Submission Checklist

Project idea.

Professional project proposal.

Requirements analysis.

Functional requirements.

Non-functional requirements.

Agile Scrum process model.

Project schedule.

Project milestones.

Project deliverables.

Gantt chart.

Data Flow Diagram.

Use Case Diagram.

Activity Diagram.

Entity Relationship Diagram.

Class Diagram.

Object Diagram.

Java GUI.

At least five classes from the Class Diagram implemented.

SQLite database.

Testing plan and results.

Risk analysis.

Future improvement plan.

README.md.

GitHub repository.

27 Conclusion

The Smart Student Task & Assignment Tracking System is designed as an individual software engineering project that addresses a practical academic management problem.

The project combines requirements engineering, Agile project management, UML modelling, object-oriented Java development, graphical user interface design, database management, testing, and documentation.

The use of Java 26, Swing, FlatLaf, SQLite, Maven, and IntelliJ IDEA provides a suitable technology foundation for developing the initial desktop version.

The proposed Agile development process allows the project to be developed incrementally while maintaining opportunities for testing and improvement. The UML diagrams provide a structured representation of the system before and during implementation.

The final system will provide students with a centralized platform for organizing academic tasks, assignments, deadlines, and progress.

Smart Student

Plan your work. Track your progress. Achieve your goals.

Student Information

Name:	Md. Mashrafi Rahman
Student ID:	IT-24034
Course:	SEPM-ICT-3107
Project:	Smart Student Task & Assignment Tracking System
Development Model:	Agile Scrum
Language:	Java 26
Database:	SQLite